Click-to-Source 3D

Stage 6 Report — MCP Server

Exposing Provenance to Coding Agents

---

1. Executive Summary

Stage 6 exposes the resolution engine as tools an AI coding agent can call over the Model Context Protocol. It shipped in two phases and one correction, across three commits.

The stage began with an architectural question that reshaped the plan: an MCP server runs as a separate Node process with no browser, no scene graph, and no click. The originally scoped tool surface assumed otherwise.

The investigation found the split is cleaner than expected. Source-derived provenance — the JSX stamps, the file contents, the editable literals — lives on disk and needs no browser at all. Runtime provenance — which instance, at what transform — lives only in the running page. That produced two phases: four file-based tools that work standalone, then three bridge tools that reach the live scene.

Two things were dropped or corrected during the stage rather than shipped:

`search_by_seed` had no seed to query. Seed capture remains unbuilt, documented in the plan's future section.

`apply_instance_edit` was proposed and abandoned after checking whether it could exist. Instance provenance has been read-only by construction since Stage 5, because an instance's position comes from a seeded RNG inside a placement loop — there is no literal in source corresponding to "the tree at x=41.2". The tool had no target to write.

Final surface: eight tools, 143 tests, bridge verified against a live browser.

---

2. The Architectural Problem

The panel resolves provenance because it has three things an MCP server does not: a click, a live scene graph, and the in-memory capture registry.

An agent calling a provenance tool has none of them. So the first question was not which tools to build but what a separate process can actually reach.

Reachable without a browser:

- Source files on disk, via the existing Node-side infrastructure
- The `__ctsSource` stamps, since they are written into source by the build transform
- The Babel editor, which already runs in Node

Not reachable:

- `selectedObject` and `instanceId`, which are browser state
- The capture registry, which is populated by prototype patches inside the page
- Anything requiring a raycast

One finding simplified the second list considerably. `mesh.getMatrixAt(instanceId)` reads the GPU-side instance buffer directly, so per-instance transforms do not require the capture registry — they require only a live mesh. That moved instance transforms from "needs the registry" to "needs any browser connection," which is a weaker and more tractable requirement.

---

3. Phase 1 — File-Based Tools

Four tools that work with no browser running.

Tool	What it does
`get_source`	Reads a file region from disk
`edit_parameter`	Rewrites a literal via the existing Babel editor
`list_provenance`	Scans stamped source for provenance sites
`search_by_generator`	Filters those sites by generator function

`search_by_generator` turned out to be a stamp scan rather than a registry query — the stamps are in source, so the search is static. That was a correction to the original framing, which assumed it needed the runtime index.

3.1 The edit guard

`edit_parameter` writes source based on coordinates an agent supplies. The panel never does this — it always resolves a `SourceRef` first, so line and argName come from real provenance.

Two options were weighed: require a prior resolve in the same session, or rely on the allowlist and root containment alone.

The allowlist won, for a specific reason. `editSource` already refuses ambiguous matches with `AMBIGUOUS_LOCATION`, so it will not guess at a target. The residual risk is therefore a *correct* edit to the *wrong file*, which is a scope problem, and scope is what `writableExtensions` and root containment already bound. A session-cache requirement would have added ceremony without closing the real gap — and would have broken `list_provenance` as a discovery path, since an agent must be able to find sites before editing them.

3.2 The reserved tool

`resolve_at_point` was added in Phase 1 as a terminal typed error rather than left absent, so an agent calling it receives a legible explanation — object-level provenance needs the browser bridge, and retrying will not help — instead of an unimplemented-tool failure.

---

4. Phase 2 — The Scene Bridge

Three tools that reach the running page, and the transport that makes them possible.

Tool	What it does
`list_instances`	Enumerates instanced meshes and their counts in the live scene
`resolve_at_point`	Resolves an address to per-instance provenance
`get_scene_state`	Reports what the bridge can currently see

4.1 Why Phase 2 is load-bearing rather than additive

`list_provenance` returns two sites for the dogfooding application: Terrain and Water.

That is correct, and it is a direct consequence of Stage 5. Removing the hand-maintained `instanceSourceRefs` arrays moved instanced provenance out of source entirely and into runtime capture. The panel sees it because the panel is in the browser.

Without the bridge, an agent sees two hand-tagged meshes and has no view of the rest of the scene at all. Phase 2 is the difference between an agent seeing a fraction of the project and seeing the whole of it.

4.2 The address scheme

`mesh.uuid` was the obvious handle and the wrong one. Recorded HMR data shows uuids churning from 8 to 11 on a single edit, so an agent holding an id across a reload holds a stale one.

The address is `{file, function, line, ordinal}` instead. The property that matters is that a wrong address *fails to resolve* rather than resolving to something plausible — a well-formed uuid can match the wrong element, whereas a structural address either matches or does not.

`ordinal` is required rather than defaulted. A stale two-mesh address with an omitted ordinal would silently bind to the first match; requiring it means the address fails instead. That cost a breaking change to a surface that had not shipped, which was free at the time and would not have been later.

4.3 Bridge lifecycle

The states were enumerated deliberately, because the naive implementation has an ordering bug.

`close()` on HMR dispose fires *after* the new page connects. A hub that tears down on dispose would therefore have the new page's connection destroyed by the old page's cleanup — something that works in testing and fails on the third reload.

Two pages open resolves as last-connect-wins with a warning, matching what the panel already does: one store, one selection. Rejecting a second connection would break the ordinary reload path, since reloads briefly have two.

---

5. What Verification Found

The bridge tools passed unit tests. Nine of them exercised a bridge no browser had ever connected to, and that was stated as a gap rather than counted as coverage.

Driving it live found a bug the unit tests could not have caught.

A single tab counted as two pages, so every query returned `ambiguous`. StrictMode mounts the effect twice, and the first `EventSource`'s close is not always visible to the server before the second opens — so a hub keyed by socket sees two pages where there is one document.

It is now covered by tests with negative controls that fail on revert. It was invisible because no test existed for it, not because a mock papered over it.

The stale-generation path was verified end to end, not inferred: generation 1, reload, generation 2, `STALE_GENERATION` on the held address, re-list, resolves. That is the recovery path working — an agent gets a legible instruction to re-list rather than a silent miss.

Panel and MCP agree. Instance 199 resolved to identical values through both surfaces, closing the loop between what a human sees on click and what an agent receives.

6. Two Corrections Worth Recording

Both are cases where a claim was written down and then found to be false before it shipped.

A fabricated bug in a commit message. A proposed message described an off-by-one in ordinal assignment, caught by a live two-mesh scene and invisible to unit tests because the mock always resolved. None of that happened: `bridgeClient.ts` had no git history, the ordinal code was written once and never changed, and no test in the suite mocks a resolver at all. The message was rewritten around the bug that did occur — the StrictMode double-page count.

A comment contradicting its own commit. The `generation` field's doc comment claimed a mismatch means "the world rebuilt underneath you is never silent." Measurement showed generation resets to 1 on reload and does not move on regeneration, so there is no mismatch to observe. The commit message recorded the true behaviour while the code comment next to it asserted the opposite. Fixed as its own commit, and the corrected comment states outright that this is not a staleness check — because reading it as one is exactly how a caller ends up trusting a stale address.

7. What Was Not Built

`search_by_seed` — no seed exists to query. Seed capture is documented as future work; it needs a hook at generator entry and does not vary per instance, so it needs no routing.

`apply_instance_edit` — dropped after checking whether it could exist rather than designing a guard for it. Instance provenance is read-only by construction, enforced in the resolver in two places and honoured by the panel, because per-instance values come from RNG draws and terrain samples with no corresponding literal in source.

This is the same structural limit as `variant`, from the opposite direction: `variant` cannot be *read* because it never enters the transform; an instance's position cannot be *written* because it never came from a literal.

Had the tool been writable, the generation check alone would not have sufficed — it would have needed list-membership within the current generation plus the count check, since a valid-format address can match the wrong mesh and a generation match does not imply an unchanged instance count.

`edit_generator_parameter` is recorded as a named follow-up: click an instance, edit the generator's parameter, honest that changing `noiseFloor` moves every tree rather than one. It is `edit_parameter` with an address-to-line lookup in front, and its value depends on the address scheme Phase 2 has now established.

---

8. Known Limits

No staleness detection. A held address can resolve silently to a different object if the scene regenerates. The generation counter does not help — it resets to 1 on reload and does not move on regeneration. This is documented in three places rather than fixed; fixing it requires a counter that survives reload and moves on regeneration, which is new design rather than an adjustment.

Hidden tabs report as disconnected. The bridge mounts only while the page is painting, so a backgrounded tab is indistinguishable from no page open. Recorded next to the wiring, since it is the first thing that bites anyone driving this headlessly.

A reload-during-call race, left documented rather than fixed. A `STALE_GENERATION` on a subsequent call recovers it, and hardening a sub-second window costs more than it returns.

---

9. Final State

Three commits in click-to-source and one in the consuming application, both repos in sync.

Commit	
`1b4f33f`	Phase 1 — four file-based MCP tools
`d4c4fad`	Phase 2 — the scene bridge
`f777b54`	The generation comment correction
`32335b0`	Consumer wiring, in the sibling repo

Eight tools, 143 tests, bridge verified against a live browser with panel and MCP agreeing on the same instance.

---

10. Assessment

The stage's most useful output was a question asked early: what can a process with no browser actually reach. Answering it first turned a tool list into two phases with a clean boundary, and made it obvious which tools were viable and which were assumptions.

Three things were removed rather than built — `search_by_seed`, `apply_instance_edit`, and a fabricated bug in a commit message. Each was caught by checking a claim rather than accepting it, which is the same pattern that produced the useful findings in Stages 4 and 5.

The live verification earned its place. Unit tests passed on a bridge no browser had connected to, and driving it for real found a bug that only exists when a real page mounts twice under StrictMode. That is the second time in this project a feature has passed its tests and failed its first contact with the running application.

`list_provenance` returning two sites is the honest picture rather than a gap. Static provenance genuinely is Terrain and Water; everything instanced lives in runtime capture, which is precisely what the bridge exists to reach.